Zenith: learning from failures in a reasoning graph

Will Cook • 13 October 2026

Abstract. Zenith studies how a forecasting programme can learn which distinctions its reasoning must preserve and investigate. Lab reads several views of a shared system, tests provisional hypotheses and commits forecasts. Oracle compares those forecasts with later observations and traces failure hypotheses through the saved intermediate records. Evolve proposes one reusable change. A local replay tests its effect on the computation; separate later cases test whether that change carries forward. We implement this loop with typed reasoning records and a historical financial panel. The immediate experimental object is an inspectable sequence from error to tested repair. Neither a plausible diagnosis nor a successful replay establishes general predictive improvement.

1. The learning problem. An error can have different origins. The required evidence may never have been observed. A reader may discard a useful distinction when compressing its input. A synthesis may combine dependent observations as if they were independent, or turn an association into an unsupported conclusion. A numerical model may fail although the reasoning around it was defensible. Some outcomes remain uncertain under every feasible use of the available evidence.

These possibilities require different responses. More persuasive retrospective prose does not distinguish them. Zenith records each step's inputs and explicit claims so that a proposed repair can name a particular computation, the evidence available there, and the downstream change expected from revising it. These are structured assertions made by the programme, not access to a model's private internal reasoning.

The shared-universe task. At cutoff t, forecast the five-market-session return of every basket from the histories of all baskets and all their constituents. Our panel uses published NEPSE instrument and index data [1]. It contains 147 eligible instruments formed into six research baskets, with 17 market-index histories. Membership is fixed on 29 December 2021 using preceding coverage, volume and beta; each basket holds fixed shares after equal initial-value allocation. These are constructed research indices, not historical sectors or traded ETFs. Missing quotes retain the last observation and expose their age. For basket value V_b, the target is $r_{b,t} = V_b(t+5)/V_b(t) - 1$.

Different views retain constituent returns, breadth, index context, volume, lagged relationships and quote quality. The numerical history stays on disk; Seed receives its inventory and readers receive bounded views. A flat basket average may conceal opposing constituent movements. A finding elsewhere can make that distinction worth retrieving in a second pass. A compact answer after the question is known does not establish that the useful question was cheap to discover. Whether this structure helps on financial data is an empirical question; it does not prove that a single adaptive model could not do the same work.

The source licence explicitly covers the dataset and accompanying documentation. The pinned publisher snapshot is retrospective: release timestamps and adjustment vintages are not certified point-in-time records. Prices and volume omit many possible causes, and pretrained-model knowledge remains a limitation. The task supports a research replay, not a trading-performance claim.

2. A typed reasoning programme. The financial task profile retains the native graph engine and the full Lab, Oracle and Evolve node identities. It replaces the earlier domain prompts and single-causal-thesis requirement with empirical investigation contracts. Lab has six feed nodes and nineteen reasoning nodes. Six first-pass readers feed synthesis, orientation and a frozen CP1 plan. Six directed readers then test that plan; a second synthesis feeds Director's CP2 commitment and Integrity's audit.

Every reasoning node emits a versioned envelope: node identity, stage, reasoning and a typed result. Its reasoning contains claims, distinctions and uncertainties. Claims identify their kind—observation, hypothesis, inference, limitation, decision or diagnosis—and select evidence IDs bound by code to exact input fields and value hashes; the provider receives the same constrained output schema. A distinction gives a question and at least two alternatives, with support or an unresolved status. Claim identities persist as node and local ID. Stated certainty is an epistemic label, not a calibrated probability.

Stage	Typed result	Mechanical check
Read and synthesise	Claims, distinctions, questions	Input references resolve
Plan (CP1)	Target, source, operation, lag, reason	Available queries; all targets covered
Investigate	Candidate ID, verdict, evidence	Candidate matches the executed test
Decide (CP2)	Selected IDs and rationale	Exact numerical values; complete vector
Diagnose	Node, category, evidence, feasible test	Subject record remains unchanged
Repair	Node, instruction, expected effect	Frozen policy; named replay descendants

Schema validity makes records comparable and rejects malformed references. It cannot certify that an inference follows from its evidence. Malformed responses are preserved and stop execution; there is no automatic retry heuristic. Input contract changes require a separately identified run.

Investigation and commitment. A common ridge model uses all available features and at most 252 mature training origins. CP1 chooses six investigations, one per target; each can use another basket, all baskets or the market indices. Operations select trend, breadth, volume or lagged features. Numerical code fits candidates and reports their gain against the baseline on eight earlier rolling validation origins. Each fit excludes labels that had not matured at its own cutoff.

The directed pass interprets those results against the original evidence. CP1 freezes what was proposed, not what must ultimately be believed. CP2 may reject an investigated hypothesis. The model selects candidate IDs; code owns their target mapping and forecast values. Unselected targets keep the numerical baseline, so uncertainty cannot disappear by omitting a difficult basket.

3. Failure decomposition. After CP2 is committed, Oracle opens later observations. Deterministic tools calculate each prediction error. Six reviews compare the appropriate source view with the saved Lab claims. Every review also receives Lab's topology, node instructions, typed contracts and reconstructible original inputs. A combined review, attribution, critique and repair proposals follow. Oracle distinguishes unavailable evidence, discarded evidence, inference error, numerical-model error and irreducible uncertainty; it can also report no supported failure attribution.

A wrong forecast alone does not establish a defective reasoning step. A useful diagnosis names the first supported failure, cites its saved input or output, and proposes a test feasible with information available there. An omitted news event discovered afterwards cannot be smuggled into a repaired earlier input. The graph records dependencies between computations, not economic causality.

4. Evolve and the test of a repair. Evolve selects a failure hypothesis and writes one reusable instruction, bounded to 512 characters, using the selected node's original inputs, output and task. It stages a candidate policy; source dossiers are not automatically changed.

A local replay freezes unaffected artifacts and executes the changed node and its descendants. An empty-instruction replay controls for repeated execution. This tests a computational change conditional on the saved programme and case, not a cause of the external market movement. Because the outcome helped select the repair, replay is development evidence. Later comparisons must freeze the instruction and commit both forecasts before opening outcomes. Changes, no-ops, regressions and failed attempts remain visible.

Execution evidence. The February case completed all 25 Lab, 28 Oracle and three Evolve nodes with GPT-4.1 mini via OpenRouter. Lab predicted gains for all six baskets; all six fell. B04's forecast was $+1.524\%$, against -3.999% realised. Oracle identified weak candidate-validation gains as a possible inference failure, but also suggested tests requiring unavailable data. Those suggestions are not information-feasible repairs.

Evolve proposed `freeze_selected_candidates`. Original, control and repaired forecasts had identical mean squared return error, 0.0013997011. Each replay reran Director and Integrity and preserved 23 artifacts byte for byte. An engineering-agent clarification of Evolve's role produced a longer, case-specific instruction, again without a local forecast change. Neither was adopted. This clarification was external to the experimental model. An explicit reviewer instruction to reject nonpositive validation gain also left forecasts unchanged: Director still selected Q1 with gain -0.0000204080. This is an observable failure to apply the instruction, not evidence of a useful repair.

A later May reader stopped after emitting an observation without a citation: the provider schema allowed an empty list while validation required evidence. This input-contract failure was preserved without retry. Independent later attempts are reported in `docs/status.md`. Earlier interface failures and the smaller basket pilot remain separate evidence. Execution, valid structure and a useful learned repair are different claims.

Scope and next experiments. The loop makes a real error, its diagnosis, a proposed instruction and measured replay effects inspectable. The present results also expose weak attribution and repair generation. More cases are needed to assess whether diagnoses reliably select useful repairs. Baskets within a date share market conditions; they are not independent replications. Larger chronological comparisons should include repeated unchanged controls, an all-universe numerical forecaster, and a single adaptive controller with the same observations and tools. A serial programme can execute the same DAG; exclusive expressive power is not a claim.

DSPy composes language-model modules [2]; MIPRO optimises instructions and demonstrations across programmes [3]. Prompt optimisation itself is not novel. Zenith's research focus is the connection between preserved reasoning distinctions, located failure hypotheses and information-feasible repair tests. Schemas provide an experimental vocabulary; their scientific value depends on what the resulting comparisons establish.

References

[1] SocrateAI. *NEPSE Open Data*, pinned source `497e27238a10`. Dataset-specific MIT-form licence; copyright 2025 SocrateAI.

[2] O. Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *ICLR*, 2024. arXiv:2310.03714.

[3] K. Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. *EMNLP*, 2024. arXiv:2406.11695.